

Inteligência Artificial

Modules, Packages, PIP

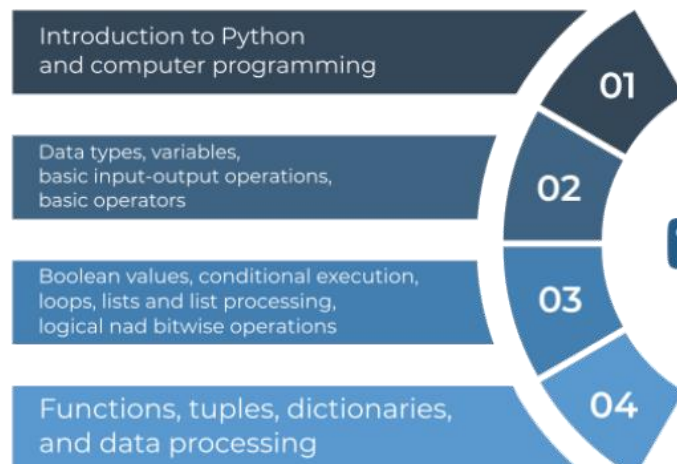
- **Disclaimer**

- These slides and notes are based on the contents of the courses:

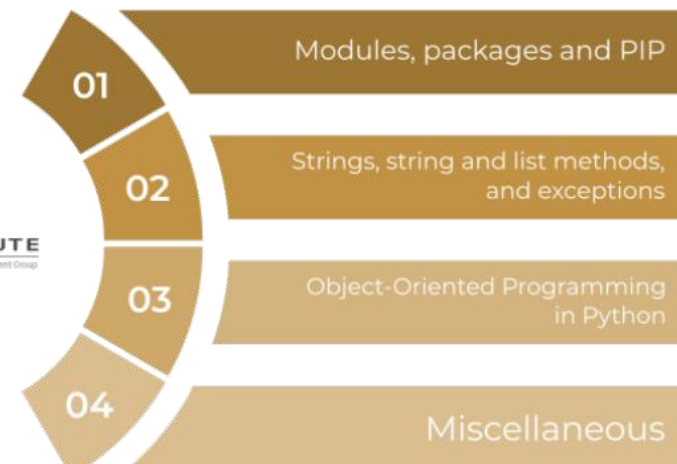
- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

Python Essentials 1



Python Essentials 2



- **Disclaimer**

- These slides and notes are based on the contents of the courses:

- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

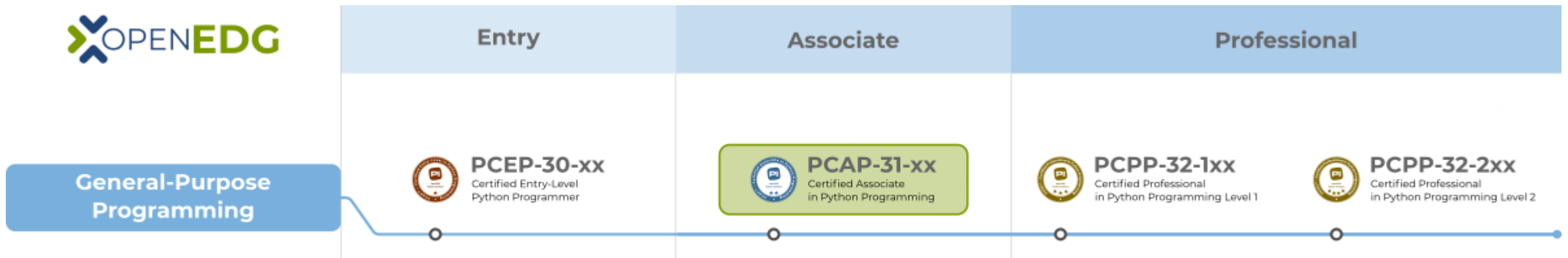


Table of Contents

- **Modules**

- Definition
- Import
- Namespace
- `dir()` function
- `sys.path`

- **Standard Modules**

- random module
 - `random()`
 - `seed()`
 - `randrange()`
 - `randint()`

- `choice()`
- `sample()`
- `random.shuffle()`

- **User Modules**

- **Packages**

- Definition
- Example

- **Python Package Installer (PIP)**

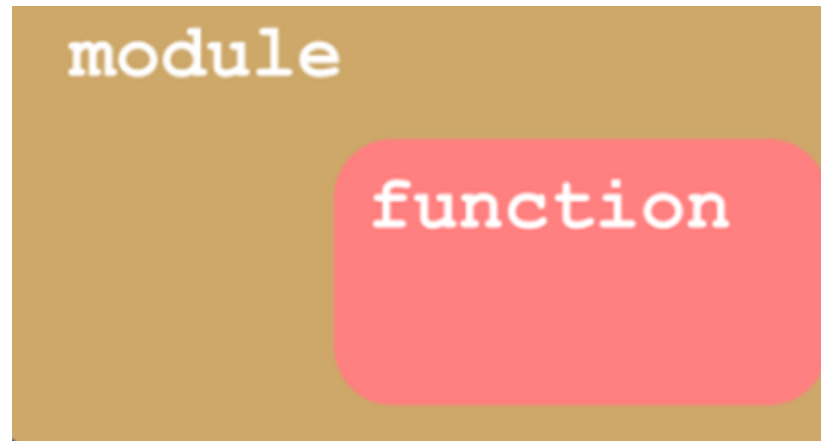
- pip
- PyPI (Python Package Index)

Python Language Programming

- **Modules**

- **Definition**

- a file (container) enclosing entities (functions, constants, etc.), which can be later imported and used when necessary
 - <https://docs.python.org/3/tutorial/modules.html>



Python Language Programming

- Modules
 - Import

```
import os  
#import namespace of module os
```



```
os.curdir  
os.listdir("Desktop")
```

```
import os as O  
#import namespace of module os with an alias O
```



```
O.curdir  
O.listdir("Desktop")
```

```
from os import *  
#import the entire namespace
```



```
curdir  
listdir("Desktop")
```

```
from os import curdir as CC  
from os import listdir as LS  
#import a specific element of the module
```



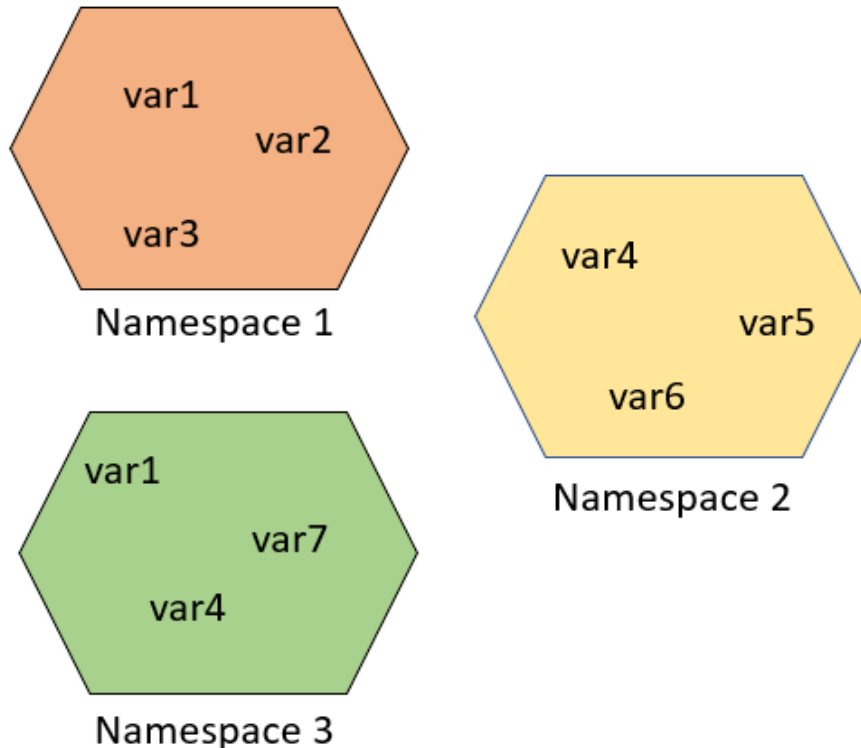
```
CC  
LS("Desktop")
```

Python Language Programming

- Modules

- Namespace

- a space in which some names exist, and the names don't conflict with each other



```
import math

def sin(x):
    if 2 * x == pi:
        return 0.99999999
    else:
        return None

pi = 3.14

print(sin(pi/2))
print(math.sin(math.pi/2))
```

two namespaces (ours and the module's one) can coexist! ESTGOH

Python Language Programming

- **Modules**

<https://docs.python.org/3/py-modindex.html>

- **dir()** function
- returns an alphabetically sorted list containing all entities' names available in the module

```
luisamveloso@DESKTOP-3MVAC0D:~$ python3
Python 3.10.12 (main, Jul 29 2024, 16:56:48) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import math
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
>>> █
```


Python Language Programming

- **Modules**

- `sys.path`
- accesses the variable **PATH** which stores the locations (folders/directories) that are searched in order to find a module

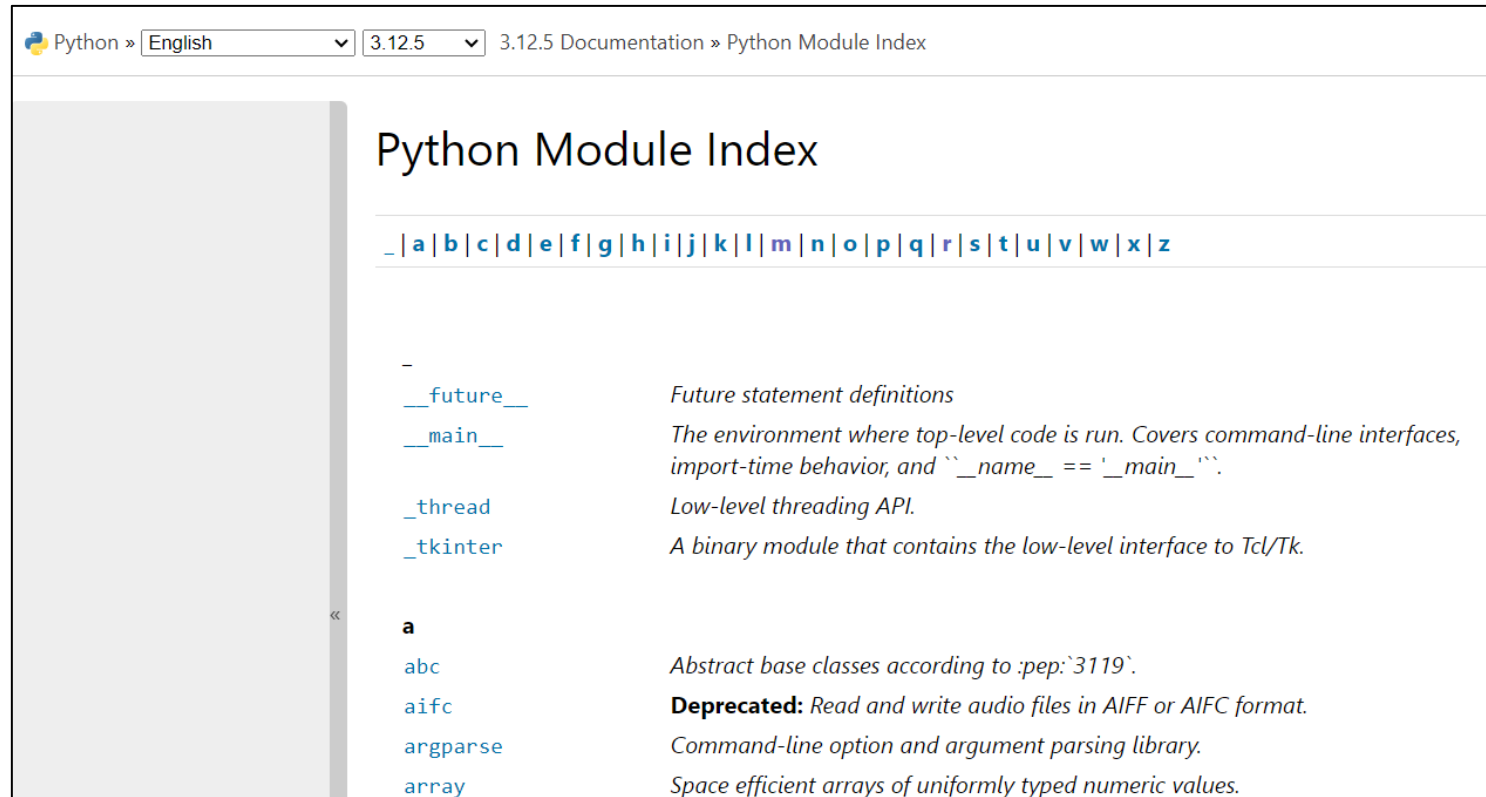
```
import sys
for p in sys.path:
    print(p)
```



```
C:\Users\user
C:\Users\user\AppData\Local\Programs\Python\Python36-32\python36.zip
C:\Users\user\AppData\Local\Programs\Python\Python36-32\DLLs
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib
C:\Users\user\AppData\Local\Programs\Python\Python36-32
C:\Users\user\AppData\Local\Programs\Python\Python36-32\lib\site-packages
```

Python Language Programming

- Modules
 - Standard Modules



<https://docs.python.org/3/py-modindex.html>

Python Language Programming

- **Modules**
 - **Standard Modules**
 - **random module**

r	
random	Generate pseudo-random numbers with various common distributions.
re	Regular expression operations.
readline (Unix)	GNU readline support for Python.
reprlib	Alternate repr() implementation with size limits.

Python » English » 3.12.5 » 3.12.5 Documentation » Python Module Index

Python Module Index

[_](#) [a](#) [b](#) [c](#) [d](#) [e](#) [f](#) [g](#) [h](#) [i](#) [j](#) [k](#) [l](#) [m](#) [n](#) [o](#) [p](#) [q](#) [r](#) [s](#) [t](#) [u](#) [v](#) [w](#) [x](#) [z](#)

-	
__future__	Future statement definitions
__main__	The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and ``__name__ == '__main__'``.
_thread	Low-level threading API.
_tkinter	A binary module that contains the low-level interface to Tcl/Tk.
a	
abc	Abstract base classes according to :pep:`3119`.
aifc	Deprecated: Read and write audio files in AIFF or AIFC format.
argparse	Command-line option and argument parsing library.
array	Space efficient arrays of uniformly typed numeric values.

Python Language Programming

- Modules

- Standard Modules

- random module

- **random(x) :**

- produces a float number x coming from the range $(0.0 \leq x < 1.0)$

```
from random import random
for i in range(5):
    print(random())
```



```
0.42956033574079777
0.9106713401132043
0.9674623113271612
0.4858902171498225
0.3091857640236457
```

Python Language Programming

- Modules

- Standard Modules

- random module

- **seed()** :

- able to directly set the generator's seed. Two of its variants:

- **seed()** - sets the seed with the current time;

- **seed(int_value)** - sets the seed with the integer value `int_value`

```
from random import random, seed
seed(0)
for i in range(5):
    print(random())
```



```
0.8444218515250481
0.7579544029403025
0.420571580830845
0.25891675029296335
0.5112747213686085
```

Note: since the seed is always set with the same value, the sequence of generated values always looks the same.

Python Language Programming

- Modules
 - Standard Modules
 - random module
 - to generate integer random values:
 - `randrange(end)`
 - `randrange(beg, end)`
 - `randrange(beg, end, step)`
 - `randint(left, right)`

```
from random import randint
for i in range(10):
    print(randint(1, 10), end=',')
```



4,10,5,9,4,4,3,5,10,3,

Python Language Programming

- Modules

- Standard Modules

- random module

- to generate unique integer random values:

- `choice(sequence)`

- `sample(sequence, elements_to_choose)`

```
from random import choice, sample

my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(choice(my_list))
print(sample(my_list, 5))
print(sample(my_list, 10))
```



```
5
[5, 8, 10, 9, 4]
[8, 4, 7, 6, 2, 3, 5, 9, 1, 10]
```

Python Language Programming

- **Modules**

- **Standard Modules**

- **random module**

- to take a sequence, like a list, and reorganize the order of the items:
 - **`random.shuffle(sequence)`**

Note: This method changes the original list, it does not return a new list

```
import random

mylist = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(mylist)

print(mylist)
```

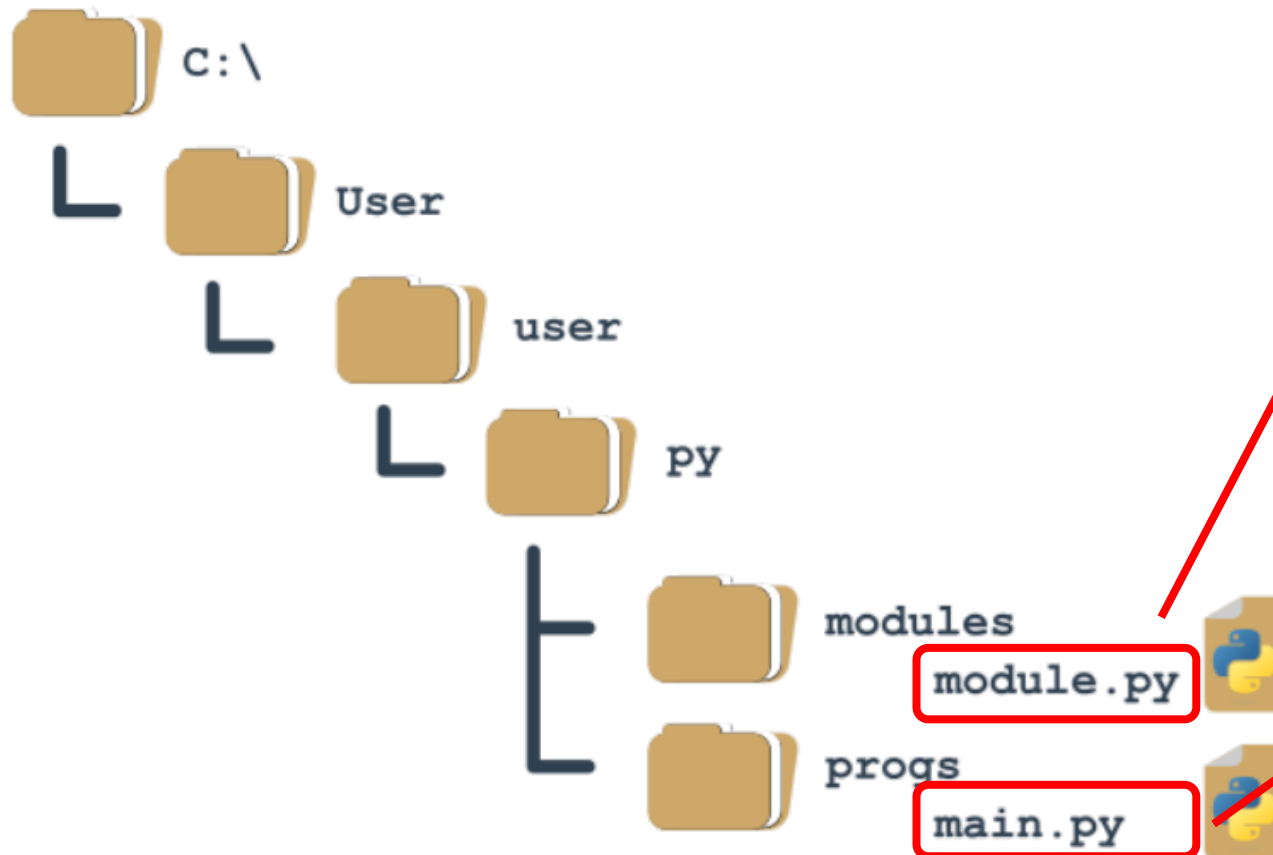


[8, 9, 4, 7, 5, 1, 3, 10, 6, 2]

Python Language Programming

- Modules

- User Modules



```
#!/usr/bin/env python3
""" module.py """
__counter = 0
def sum1(the_list):
    global __counter
    __counter += 1
    the_sum = 0
    for element in the_list:
        the_sum += element
    return the_sum
```

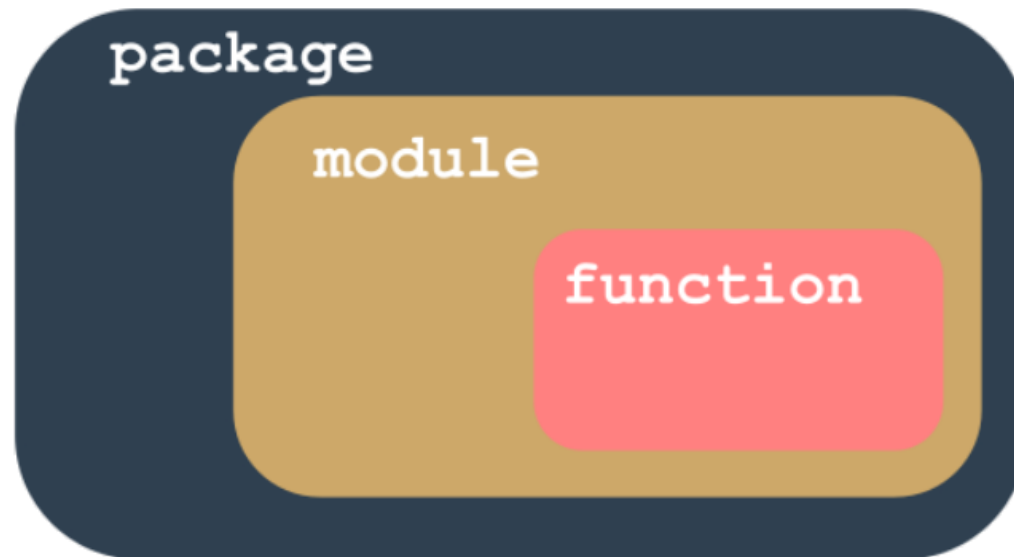
```
""" main.py """
from sys import path
path.append('../\modules')
import module
zeroes = [0 for i in range(5)]
print(module.sum1(zeroes))
```

Python Language Programming

- Packages

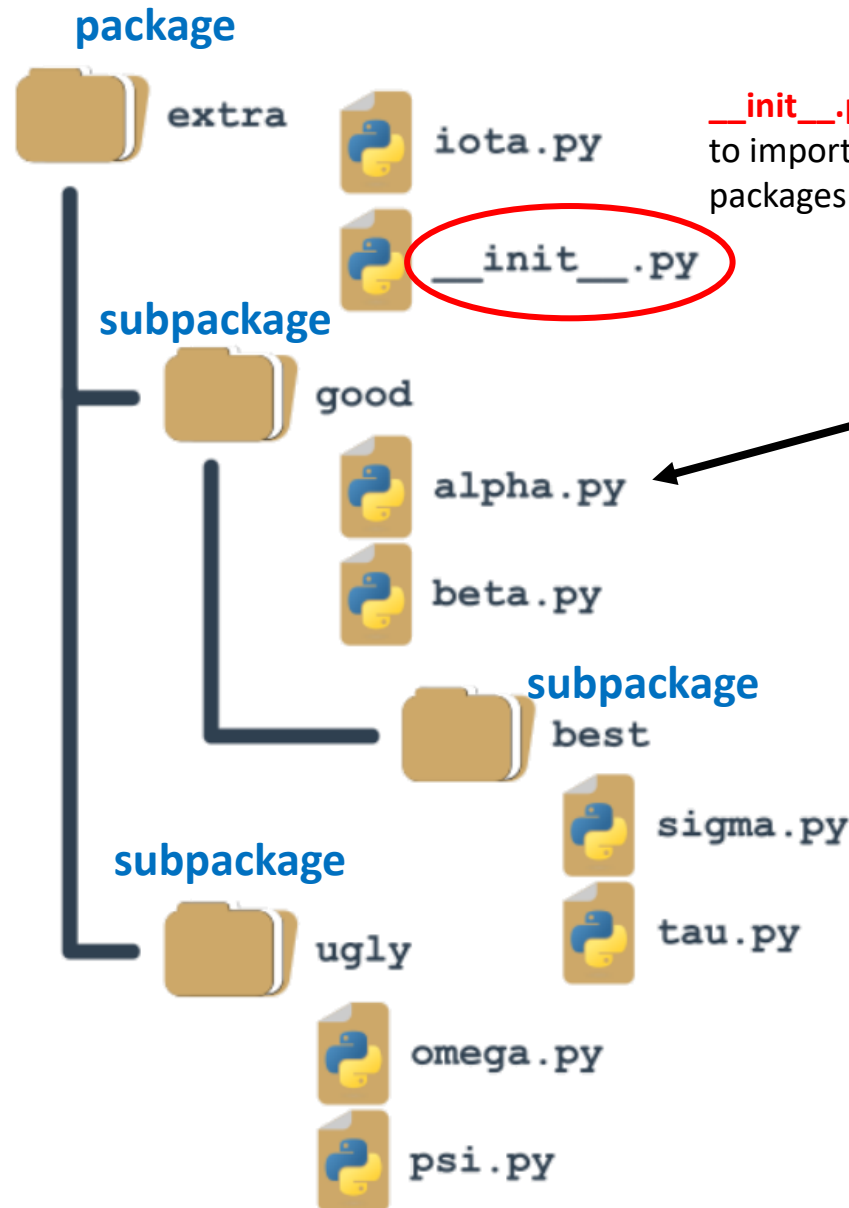
- Definition

- a package plays a similar role to a folder/directory in the world of files
 - a kind of container filled with modules



Python Language Programming

- Packages
- Example



`__init__.py` : implicitly run when a package containing it is subject to import and is used **to initialize a package** and/or its sub-packages (if any). The file may be empty, but must not be absent

```
#!/usr/bin/env python3
""" module: alpha """
def funA():
    return "Alpha"
if __name__ == "__main__":
    print("I prefer to be a module.")
```

`__name__`

- when you run a file directly, this variable is set to `__main__`;
- when a file is imported as a module, this variable is set to the file's name (excluding .py)
- `_` or `__` instructs that an entity should be treated as private, but this is only a recommendation, not an order.

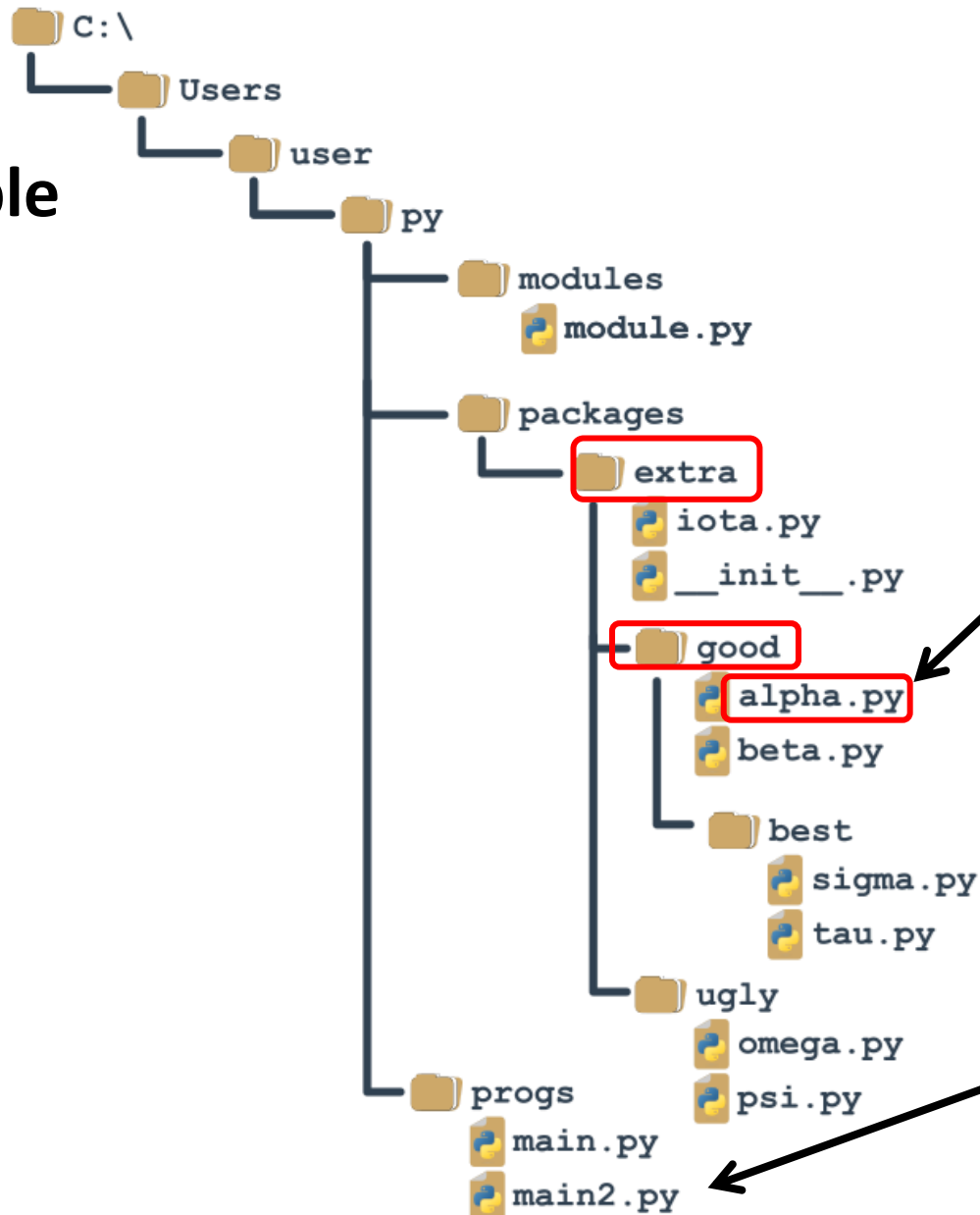
`#!`

- denominated shebang, hasbang, poundbang
- used to instruct Unix-like OSs how the Python source file should be launched. This convention has no effect under MS Windows.



Python Language Programming

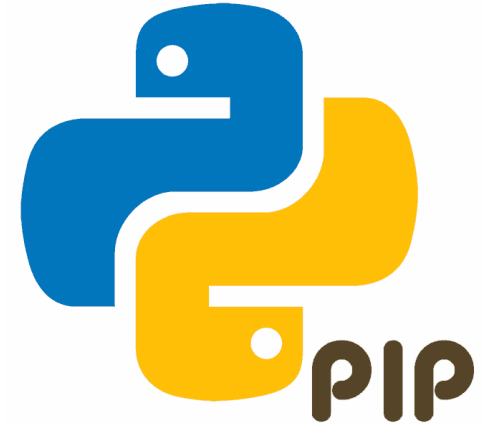
- Packages
- Example



```
#!/usr/bin/env python3
""" module: alpha """
def funA():
    return "Alpha"
if __name__ == "__main__":
    print("I prefer to be a module.")
```

```
from sys import path
path.append('../\packages')
import extra.good.alpha
print(extra.good.alpha.funA())
```

Python Language Programming



- **Python Package Installer (PIP)**

- **pip**

- means “*pip installs packages*”

- `pip3 --version`
 - `pip help operation` - shows brief pip's description;
 - `pip list` - shows list of currently installed packages;
 - `pip show package_name` - shows package_name info including dependencies;
 - `pip search package_name` - searches through PyPI directories;
 - `pip install package_name` - installs name system-wide;
 - `pip install --user package_name` - install package for a specific user;
 - `pip install -U package_name` - updates previously installed package;
 - `pip uninstall package_name` - uninstalls previously installed package;

Python Language Programming

- Python Package Installer (PIP)
 - PyPI (Python Package Index)
 - Python repository
 - <https://pypi.org/>
 - maintained by a workgroup named the *Packaging Working Group*, a part of the Python Software Foundation
 - sometimes referred to as the *Cheese Shop* (Monty Python sketch)



<https://www.youtube.com/watch?v=Hz1JWzyvv8A>

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from <https://www.netacad.com/>**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.